This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCC.2017.2739146, IEEE Transactions on Cloud Computing

Toward Practical Privacy-Preserving Frequent Itemset Mining on Encrypted Cloud Data

Shuo Qiu, *Student Member, IEEE,* Boyang Wang, Ming Li, *Member, IEEE,* Jiqiang Liu, and Yanfeng Shi

Abstract—Frequent itemset mining, which is the essential operation in association rule mining, is one of the most widely used data mining techniques on massive datasets nowadays. With the dramatic increase on the scale of datasets collected and stored with cloud services in recent years, it is promising to carry this computation-intensive mining process in the cloud. Amount of work also transferred the approximate mining computation into the exact computation, where such methods not only improve the accuracy also aim to enhance the efficiency. However, while mining data stored on public clouds, it inevitably introduces privacy concerns on sensitive datasets.

In this paper, we propose a new framework for enforcing privacy in frequent itemset mining, where data are both collected and mined in an encrypted form in a public cloud service. We specifically design three secure frequent itemset mining protocols on top of this framework. To guarantee data privacy and computation efficiency, we adopt two different homomorphic encryption schemes and design a secure and effective comparison scheme. Our first protocol achieves more efficient mining performance while our second protocol provides a stronger privacy guarantee. In order to further optimize the performance of the second protocol, we leverage a minor trade-off of privacy to get our third protocol. Finally, we evaluate the performance of our protocols with extensive experiments, and the results demonstrate that our protocols obviously outperform previous solutions in performance with a similar security level.

Index Terms—Cloud computing, privacy-preserving, frequent itemset mining, efficiency.

1 Introduction

With the raise of big data, data mining has become one of the most emerging techniques for data analytics on massive datasets. As one of the fundamental operations in association rule mining, frequent itemset mining, plays a significant role in market prediction [1], intrusion detection [2], network traffic management [3] and web analysis [4]. Specifically, given a transaction database, where each tuple in this database is denoted as a transaction, frequent itemset mining is able to discover popular itemsets and their potential interesting connections from this transaction database. For instance, Netflix or RedBox can perform frequent itemset mining to discover whether people watched Fast and Furious will also watch Mission Impossible in order to recommend new-release movies to costumers.

The easy accesses and low prices of public cloud services can significantly save mining costs on massive datasets. More importantly, since large-scale data from multiple and variable data sources (e.g., millions of users and devices) are collected by cloud services, such as Google, mining on cloud data can also dramatically improve the accuracy and effectiveness of mining. On the other hand, since many datasets collected by cloud services are sensitive, such as locations, medical records and financial data, mining these confidential data inevitably brings important privacy issues. For example, a *curious* service provider can easily reveal confidential data in a transaction database while users and miners would like to keep the data in private during the entire mining process.

In order to preserve privacy of frequent itemset mining in public cloud services, some randomization-based approaches [5], [6], [7], [8], [9] have been proposed. Unfortunately, randomization-based approaches dramatically scarify the accuracy and utility of frequent itemset mining with limited privacy guarantee[1]. Moving a step forward, Yi *et al.* [10] recently proposed a privacy-preserving frequent itemset mining protocol in public clouds, where all the encrypted transactions are centralized to the cloud and miners delegate all the mining tasks to the cloud. However, to enable frequent itemset mining, this work requires n (where $n \geq 2$) aided semi-honest servers (besides the cloud server storing data) to perform distributed decryption during the evaluation on encrypted data. Moreover, the need of these additional n aided semi-honest servers slows down the running time of frequent itemset mining, and introduces huge interactions and communication overheads.

In this paper, we propose a privacy-preserving framework for secure frequent itemset mining on encrypted data, where only one aided server (referred to as Evaluator) is needed besides the Cloud Service Provider (CSP). The CSP collects encrypted transactions and maintains an encrypted transaction database, and the Evaluator assists the CSP to carry mining on encrypted data. Based on this framework, we design three protocols, where the first protocol (i.e., Protocol 1) is better in terms of mining time and the second protocol (i.e., Protocol 2) is stronger in the aspect of privacy protection. To achieve a better efficiency of Protocol 2, we introduce the third protocol via sacrificing little privacy. Due to the leverage of this only one aided server, our protocols outperform the previous work [10] mentioned above in the aspect of running time with a similar privacy level in practice. A comparison between our protocols and this previous work is presented in Table 1. In addition, we further optimize the

- *S. Qiu, J. Liu and Y. Shi are with the School of Computer and Information Technology, Beijing Jiaotong University, Beijing 100044, China. E-mail: {qiushuo,jqliu,schwannfeng}@bjtu.edu.cn.*
- *B. Wang and M. Li are with the Department of Electrical and Computer Engineering, the University of Arizona, Tucson, AZ 85721, USA. E-mail: {boyangwang,lim}@email.arizona.edu.*

1. Since the main idea of randomization-based approaches is adding noisy into the original sets, the mining results can only provide an approximate result. Therefore, all the solutions based on this method mainly aim to balance the trade-off between privacy and accuracy.

2168-7161 (c) 2017 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCC.2017.2739146, IEEE Transactions on Cloud Computing

performance of our second protocol with a minor privacy tradeoff. The details of our contributions are listed as below:

- In the protocols, we guarantee that the Evaluator obtains no extra information except the frequent results (i.e., 1s or 0s) via blinding and adding dummy sets. While for CSP, we formulate three security levels based on different leakage functions in our protocols, where *Privacy Level 1 (PL-1)* includes the mining itemset, its support and the frequent result, *Privacy Level 2 (PL-2)* contains the number of the mining items and the frequent result, and *Privacy Level 3 (PL-3)* reveals only the frequent result.
- We present three practical privacy-preserving frequent itemset mining protocols in the paper. Our Protocol 1 is extremely efficient with the *PL-1*, while our second protocol achieves a higher privacy level *PL-3*. In protocol 2, we leverage a new efficient comparison protocol to achieve more feasible efficiency. Furthermore, we also apply some anonymization methods to obtain Protocol 3, which is an enhanced version of Protocol 2 with *PL-2* privacy.
- Experimental results demonstrate that our protocols are practical and compared to the most-related previous work [10], the implementation of the results show that our protocols are more efficient than [10]'s protocols with the similar privacy level.

2 Related Work

The problem of frequent itemset mining was originally described by Agrawal *et al.* [11], [12], and has been extensively studied in recent years. Evfimievski *et al.* [13] proposed corresponding solution with association to vary randomization operators based on Apriori algorithm. However, their scheme is the estimation using Bayesian network. Oliveira *et al.* [14] propose a privacy preserving frequent itemset mining algorithm with a trade-off on privacy and accuracy. And then amount of privacy-preserving frequent itemset mining solutions have been proposed in the literature (e.g., [6], [9], [15], [16], [17], [18], [19], [20]). Generally speaking, all the existing privacy preserving solutions can be classified into two categories: randomization-based solutions and cryptography-based solutions.

Randomization-based solutions. Several studies leveraged perturbation approaches to achieve privacy-preserving association rule mining (e.g., [13], [16]). The main idea in these approaches is to sanitize all the samples prior to their release to achieve privacy preservation. Specifically, using the information on the distribution of the random data or some generalization techniques to distort the original data, thus it can generate an approximation to the original data distribution without revealing the original data values. The most popular techniques include adding "noise" into a transaction database [21], and leveraging k-support anonymity [22] which requires that each item has a similar support with at least k-1 items. Another main approach is to use k-privacy [18], which requires each itemset (a set of items) cannot be distinguished from at least k-1 itemsets with the same size. However, the randomization-based approaches have significant drawbacks, where it can only provide an approximate result and limited privacy levels.

Cryptography-based solutions. Cryptography-based methods provide a well-defined security level and an exact mining result for privacy-preserving frequent itemset mining. Several studies have been focused on this direction. [5] presented the corresponding protocols with data being vertically partitioned among different sites using secure scalar product protocols. In [6], the authors proposed a solution for the complementary situation where data is horizontally partitioned. With the prevalent of large scale datasets, both of the two solutions are not efficient to satisfy the practical requirement. [23] and [24] proposed efficient protocols by leveraging Bloom filters. However, similar to randomized-based solutions, using Bloom filters would sacrifice the accuracy of mining results. Vaidya *et al.* [7] transferred the problem of association rule mining to private set intersection, and successfully utilized a secure set intersection cardinality protocol to achieve rule mining. More recently, Dong *et al.* [9] utilized an efficient private set intersection protocol [25] to achieve a more efficient inner product protocol for evaluating association rule mining. While both two solutions are two-party protocols, which involve extensive interactions and are not feasible to the system framework we studied in this paper. And other work [26] and [20] achieve a high efficiency with using a secret sharing and secure inner product protocol respectively, while these solutions are unable to preserve the information of mining result, which would result in the leakage of the original input data. Lai *et al.* [27] provided a semantic security secure solution for outsourcing association rule mining with both data privacy and mining result privacy, while the efficiency is not enough for the practical requirements. Later, Yi *et al.* [10] also proposed a secure association rule mining scheme in the same outsourcing cloud computing environment with a distributed ElGmal homomorphic encryption, while their scheme needs extra $n(\geq 2)$ aided servers to collaborate with the cloud, and huge amounts of communication overhead are generated during this process.

3 Problem Statement

3.1 System Model

The system model of our paper is illustrated in Fig. 1. Specifically, there are n users $\{U_1, ..., U_n\}$, a data miner, a Cloud Service Provider (CSP), and an Evaluator. Each user uploads a transaction (or multiple transactions) to the CSP. The CSP maintains a transaction database, which includes a massive number of transactions contributed from different users. The data miner can submit frequent itemset mining queries to the CSP, and we leverage an Evaluator to assist the CSP to perform frequent itemset mining efficiently on the transaction database. At the end of the executions, the CSP will return a Boolean result (e.g., frequent or not) of a mining query to the data miner.

3.2 Threat Model

In our model, users do not fully trust the CSP with the privacy of their transactions. Therefore, all their transactions are encrypted before contributing to the CSP. Both the CSP and the Evaluator are *honest-but-curious*, which indicates both of them can compute data correctly by following protocols but are curious about data in the transaction database and results of frequent itemset mining. In addition, we assume that the CSP and the Evaluator *do not collude*. The data miner can submit frequent itemset mining queries *either in plaintext or in ciphertext* due to different privacy requirements (i.e., whether the data miner wants to reveal its queries to the CSP or not). Note that this non-colluding model between the CSP and the Evaluator has been extensively leveraged in many recent papers (e.g., [28], [29], [30]) to *efficiently* enable advanced computations on massive encrypted data.

2168-7161 (c) 2017 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCC.2017.2739146, IEEE Transactions on Cloud Computing

TABLE 1
Comparison among our protocols and YRB's Protocol [10].

	Model		Privacy	Efficiency	
	Servers	Miner		Computation	Interactions (Servers)
YRB's Protocol [10]	$n \geq 3$	Data Owner	Database Privacy	Slow	more than n
Our Protocol 1	two	Any Users	Item Privacy	Very Fast	One
Our Protocol 2	two	Any Users	Database Privacy	Slightly Fast	Two
Our Protocol 3	two	Any Users	Database Privacy	Fast	Two

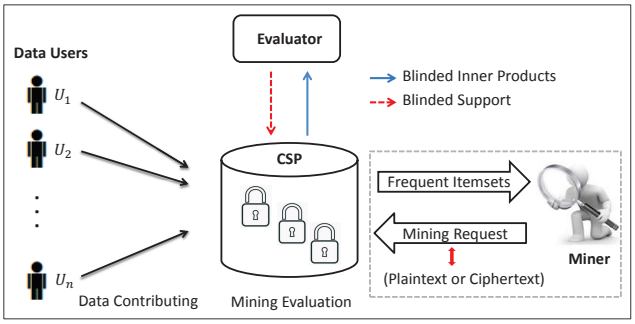


Fig. 1. System model: Multiple users and one Miner in our system. The non-colluding Evaluator and the Cloud Service Provider (CSP) together evaluate the support of the mining itemset and return mining results without leaking data privacy.

3.3 Privacy Objectives

In our paper, besides asking the CSP and the Evaluator to compute frequent itemset mining efficiently and correctly, another main objective of ours is to preserve the privacy of transactions and the mining result (i.e., the support of a query in transaction database). Informally speaking, after performing frequent itemset mining, except learning whether a query is frequent or not, the CSP or the Evaluator cannot learn the raw data in the transaction database. The content of the frequent itemset mining query should also be preserved from the CSP or the Evaluator if the data miner chooses to submit its query privately. Moreover, exact supports or confidences of the mining itemsets should be concealed as such information can be used to infer some information about the raw data. Since different privacy requirements maybe occur in our system, we hereby define three levels of privacy based on a *leakage function* $\mathcal{L}$ which captures all the information that are allowed to leak during the evaluation on the encrypted data in a protocol.

Definition 1. (*Privacy Level 1 (PL-1)*) The Miner submits a mining request in a plaintext form, which means the mining itemset is not private to CSP. Thus, at the end of the protocol, the leakage function $\mathcal{L}$ of CSP includes the mining itemset (i.e., q), the support of the mining itemset (i.e., *supp(q)*) and the mining result (i.e., *result*), which can be represented as

$$\mathcal{L}(\lambda, \bar{x})) = \{q, supp(q), result\},$$

where λ is the security parameter and $\bar{x}$ denotes the input of protocol.

Definition 2. (*Privacy Level 2 (PL-2)*) The mining itemset (i.e., q) and the support of the mining itemset (i.e., *supp(q)*) are sensitive information which should be protected from CSP. Thus once the protocol ending, only the mining result (i.e., *result*) and the number of items in the mining itemset (i.e., *itemnum(q)*) can be in the leakage function $\mathcal{L}$, which can be represented as

$$\mathcal{L}(\lambda, \bar{x})) = \{itemnum(q), result\},$$

where λ is the security parameter and $\bar{x}$ denotes the input of protocol.

Definition 3. (*Privacy Level 3 (PL-3)*) At the end of the protocol, only the mining result (i.e., *result*) are leaked in the leakage function $\mathcal{L}$ of CSP, which can be represented as

$$\mathcal{L}(\lambda, \bar{x})) = \{result\},$$

where λ is the security parameter and $\bar{x}$ denotes the input of protocol.

Real vs. Ideal Normally, two approaches are always to be used to define the security of a secure scheme, the Game-Based method and Simulation-Based method, where these two approaches has been proved to be equivalent while analyzing the security of encryption scheme [31]. In this paper, we leverage the Simulation-Based approach, which is defined with *real vs. ideal* model, to formalize the privacy of the scheme. This model also has been widely used in defining the security of multi-party computation [32] and searchable encryption [31]. Specifically, we assume that an adversary $\mathcal{A}$ interacts with a challenger (i.e., the party who has a secret key) in the real world to perform a protocol Π, while it interacts with a simulator $\mathcal{S}$ in the ideal world to complete the protocol. If the view of this adversary obtained from the real world is *(computationally) indistinguishable* from the view of it in the ideal world, then we say this protocol Π is $\mathcal{L}$-secure, which can be described as

$$\{\text{IDEAL}^{\Pi}_{\mathcal{A},\mathcal{S},\mathcal{L}}(\lambda, \bar{x})\} \stackrel{c}{\equiv} \{\text{REAL}^{\Pi}_{\mathcal{A},\mathcal{L}}(\lambda, \bar{x})\}$$

where λ is the security parameter (e.g., the length of a secret key), $\bar{x}$ denotes the input of protocol Π in general, and $\mathcal{L}$ contains all the privacy leaked to $\mathcal{A}$ as defined in the above three privacy levels. In our protocols, we can see the CSP or the Evaluator as the adversary, and the data miner as the challenger.

4 Preliminaries

In this section, we briefly introduce frequent itemset mining, and present two homomorphic encryption primitives, including Paillier [33] and BGN [34], which serve as the building blocks in our protocols to help us evaluate association rules on encrypted data. Two secure inner product protocols are also introduced and an efficient comparison protocol is described.

4.1 Frequent Itemset Mining

As we stated ahead, frequent itemset mining is one of the most significant data mining techniques, and it is extensively leveraged to discover relations between different items in massive datasets. Agrawal *et al.* [11] first presented a detail description of this conception. Specifically, let $I = \{i_1, \cdots, i_n\}$ be the set of all the items. A transaction t is a set of items, where $t \subseteq I$. A transaction dataset T is a set of transactions, where $T = \{t_1, \cdots, t_m\}$ and m denotes the total number of transactions. The purpose of frequent

2168-7161 (c) 2017 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCC.2017.2739146, IEEE Transactions on Cloud Computing

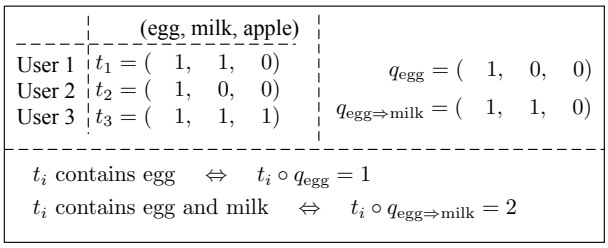


Fig. 2. An example of association rule mining, where $\circ$ denotes an inner product operation between two binary vectors, $supp(X) = 3$, $supp(X \Rightarrow Y) = 2$, $conf(X \Rightarrow Y) = 2/3$, $X = \{\text{egg}\}, Y = \{\text{milk}\}$.

itemset mining is to discover the frequency of the item/itemsets, which can further find the the relation of two items.

For instance, the set of all the items in Walmart is I, a transaction is the list of items (e.g., eggs, milk, and apples) a user bought, and the transaction dataset T is all the transactions that Walmart sold in one day. The relation between two items are normally measured by two values, *support* and *confidence*. Assume we have an itemset X, where $X \subseteq I$. The support of X, referred to as $supp(X)$, is the number of transactions containing X in the transaction dataset T, and it is a measure of statistical significance. An association rule is an implication of the form $X \Rightarrow Y$, where $X \subseteq I$, $Y \subseteq I$, and $X \cap Y = \varnothing$. The support of rule $X \Rightarrow Y$, denoted as $supp(X \Rightarrow Y)$, is the number of transactions containing $X \cup Y$ in T. If $supp(X \Rightarrow Y) \geq thresh$, the itemset $X \cup Y$ is a frequent itemset, where *thresh* is the threshold of the frequent itemsets. Once obtaining the *support* of the itemset, we can easily compute the confidence of rule $X \Rightarrow Y$, which is a measure of the relation between two items. It can be computed as

$$conf(X \Rightarrow Y) = \frac{supp(X \Rightarrow Y)}{supp(X)} = \frac{supp(X \cup Y)}{supp(X)}. \quad (1)$$

Specifically, if $X = \{\text{eggs}\}$ and $Y = \{\text{milk}\}$, $supp(X)$ indicates the number of transactions containing eggs in T and $supp(X \Rightarrow Y)$ denotes the number of transactions containing both eggs and milk in T. The rule of how likely a user buys eggs will also buy milk is evaluated via $conf(X \Rightarrow Y)$.

Generally, there will be many rules in a massive dataset. The ones with low supports and confidences clearly do not indicate useful relations. A minimum support and a minimum confidence can be used as two thresholds to identify meaningful relations. As described above, an itemset is said to be *frequent* if its support is greater than the threshold. When all the frequent itemsets are generated, all the rules can be evaluated straightforwardly by the *confidence* as indicated in Eq. (1).

Normally, items can be presented as Boolean types, i.e., an item is described as either exist or absent. Therefore, a transaction dataset T can also be presented as an $m \times n$ binary matrix, where each row is a transaction and each column describes an item[2]. Specifically, if element $e_{j,k}$ in the binary matrix is 1, it indicates transaction t_j contains item i_k; otherwise, t_j does not include i_k. As a result, finding a frequent itemset in the transaction dataset can be evaluated by computing an inner product (as shown in Fig. 2).

2. Here we suppose that the number of possible items is not very large since only limited items in the market are very popular.

4.2 Encryption Primitives

Paillier Encryption. Paillier Encryption is an additively homomorphic encryption proposed by Paillier [33]. In this paper, we mainly focus on its additive homomorphism, which means it can compute a new ciphertext of plaintext $(m_1 + m_2)$ based on the two ciphertexts of plaintext m_1 and m_2 without revealing the plaintexts. Specifically,

- Given $[m_1]$ and $[m_2]$, we can compute $[m_1 + m_2] = [m_1] \cdot [m_2]$.
- Given $[m_1]$ and a plaintext α, we can compute $[\alpha \cdot m_1] = [m_1]^{\alpha}$.

where $[m]$ describes the Paillier ciphertext of a plaintext m. More details of construction of Paillier encryption can be found in [33].

BGN Encryption. Boneh-Goh-Nissim (BGN) encryption is a somewhat homomorphic encryption, which was proposed by Boneh *et al.* [34]. With the homomorphic property, it can compute an arbitrary number of additions and also one multiplication on encrypted data. Specifically, we have

- Given $\|m_1\|$ and $\|m_2\|$, compute $\|m_1 + m_2\| = \|m_1\| \odot \|m_2\|$
- Given $\|m_1\|$ and $\|m_2\|$, compute $\|m_1 \times m_2\| = \|m_1\| \otimes \|m_2\|$

where $\|m\|$ represents the BGN ciphertext of a plaintext m, $\odot$ and $\otimes$ denote the corresponding operations in the ciphertext domain. More construction details of BGN can be found in [34].

We will use Paillier and BGN respectively for evaluating inner products with different privacy requirements, which essentially leads to secure association rule mining. Understanding the homomorphic properties of these two primitives will be sufficient to follow the design of our protocols. The details and security proofs of Paillier and BGN can be found in [33] and [34] respectively.

4.3 Securely Evaluating Inner Products

We now describe how to evaluate inner products securely on encrypted data as in [30]. To deal with different privacy requirements (whether the other vector is in an encrypted form), we offer two algorithms, where each of them can output an encrypted inner product. The first one, denoted as InnerProPC, is able to compute an inner product of two vectors, where one is in plaintext while the other one is in ciphertext. The second one, denoted as InnerProCC, can calculate an inner product of two vectors, where both of the two vectors are encrypted. The details of these two algorithms are illustrated below, where the first one is built based on Paillier, and the second one is designed using BGN.

Algorithm 1 InnerProPC

Input: Given a vector $\vec{x} = (x_1, \ldots, x_n)$ and an encrypted vector $[\vec{y}] = ([y_1], \ldots, [y_n])$
Output: Output $[\vec{x} \circ \vec{y}]$

```
temp = 1, i = 1;
while i <= n do
    temp = temp · [y_i]^{x_i};
    i++;
end while
return temp
```

2168-7161 (c) 2017 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCC.2017.2739146, IEEE Transactions on Cloud Computing

Algorithm 2 InnerProCC

Input: Given two encrypted vectors $\|\vec{x}\| = (\|x_1\|, \ldots, \|x_n\|)$ and $\|\vec{y}\| = (\|y_1\|, \ldots, \|y_n\|)$
Output: Output $\|\vec{x} \circ \vec{y}\|$
1: $\texttt{temp} = 1, i = 1$;
2: **while** $i <= n$ **do**
3: $\quad \texttt{temp} = \texttt{temp} \odot (\|x_i\| \otimes \|y_i\|)$;
4: $\quad i++$;
5: **end while**
6: **return** temp

Algorithm 3 Compare

Input: **Alice** has two ciphertexts $[a]$ and $[b]$ under Paillier encryption, where $a, b \leq 2^l$; **Bob** has (Paillier) private key SK.
Output: 1 or 0.
1: **Alice**: calculates $[x] = [b] \cdot [a]^{-1} \bmod N^2$; $\quad \triangleright\ x = b - a$
2: **Alice**: chooses a random $r \leftarrow \{1, \cdots, 2^t\}$ such that $2^{t+l} < N/2$, computes $[z] = [x]^r \bmod N^2$ and sends $[z]$ to **Bob**; $\quad \triangleright\ z = r \cdot x$
3: **Bob**: decrypts $[z]$, and gets $z = r \cdot (b - a)$;
4: **Bob**: outputs 1 if $z < N/2$, otherwise outputs 0.

4.4 Secure Comparison

Here we introduce a new efficient comparison protocol on encrypted data between two parties (e.g., Alice and Bob). Specifically, Alice has two encrypted data $[a]$ and $[b]$, where both $[a]$ and $[b]$ are encrypted with Paillier, while Bob has the secret key. Alice and Bob will run this protocol interactively in order to compare a and b but without revealing the value of a and b to neither Alice nor Bob. This protocol is essentially achieved via the additively homomorphic property of Paillier and proper blinding. Details of this algorithm is presented in Algorithm 3.

At the end of this protocol, Alice outputs 1 (i.e., $a \geq b$) if $z < N/2$; otherwise it outputs 0 (i.e., $a < b$). Note that this comparison protocol presented above is not a general approach to compare two encrypted data, it works only for plaintexts with a small size and relatively small blind factors (i.e., for a message m and a blind factor r, $m \cdot r$ should be less than $N/2$). For instance, if N is 512 bits in Paillier and m is 32 bits, then r can go up to 480 bits. However, since the size of plaintexts encrypted with Paillier in our scenarios is small (e.g., 0, 1, supports) compared to N, we leverage this comparison protocol here for efficiency purpose.

5 Secure Frequent Itemset Mining

As we stated in Section 3, the CSP holds a set of encrypted transactions, and a Miner would like to mine frequent itemsets on this encrypted transaction database. Each transaction is represented as a binary vector, and a mining query is represented as another binary vector. We say a transaction satisfies a mining query if the inner product of this transaction and this mining query is equal to the number of 1s in the mining query. The CSP and the Evaluator can interactively compute and decide the support of a mining query on a transaction database. Based on different privacy requirements, we now describe three protocols in the following part to perform frequent itemset mining on encrypted data, where the first protocol focuses on efficiency while the second one is strong in terms of privacy protection. Then, we further optimize and enhance the performance of our second protocol to get the third one with a minor tradeoff in terms of privacy.

5.1 Protocol 1

We first consider Protocol 1 where a Miner submits its mining query in plaintext while all the transactions are still encrypted. CSP receives the encrypted transactions from data users and the plaintext mining query from the Miner, and then compute the inner products of them. Therefore, the additive homomorphism Paillier encryption is enough in Protocol 1. Details of this protocol are shown in Fig. 3.

Details. Specifically, the Evaluator first generates a pair of public/private keys $\{\mathsf{PK}, \mathsf{SK}\}$ for Paillier in **Step 1**. Then, each user encrypts its transactions with the public key of the Evaluator and contributes its encrypted transactions to the CSP in **Step 2**. Given a mining query, the CSP computes an encrypted inner product of each encrypted transaction and the mining query by calling InnerProPC introduced in Subsection 4.3 as a subroutine. In order to figure out whether each encrypted inner product satisfies the requirement (i.e., the inner product is equal to the number of 1s in the mining query), the CSP interacts with the Evaluator by using blinding. Meanwhile, to hide the actual support of this mining query on the encrypted transaction database from the Evaluator, the CSP also randomly generates some dummy encrypted transactions, evaluates encrypted inner products of these dummy encrypted transactions with the mining query, shuffles these encrypted inner products with the undummied ones and sends them all together to the Evaluator. The Evaluator returns a dummied support to the CSP. Finally, the CSP can filter the result of the dummied one, and outputs the actual support of the mining query.

Discussion. After this protocol, the CSP learns the support of mining query, which makes it easy for it to decide whether it is frequent or not by comparing the support with the threshold. While for the Evaluator, it does not even know the support of the mining query. In addition, neither the CSP nor the Evaluator learns any transactions in the database. Here we give the theorem of its security below and present the details of the security proof. The security of this protocol is similar to *item privacy* presented in [10].

Theorem 1. *Our Protocol 1 is secure against the semi-honest CSP with PL-1 privacy as long as Paillier encryption is semantically secure, and also against a semi-honest Evaluator to distinguish the intermediate results of the computation for the mining itemsets as long as non-collusion is occurred between CSP and Evaluator.*

Proof: Informally speaking, the protocol described above is secure if an adversary $\mathcal{A}$ is unable to distinguish between the view generated by a challenger (who has the secret information of the executions) in the real world and the view generated by a simulator (who simulates all the view based on the leakage function $\mathcal{L}$) in the ideal world. In the following, we separately analyze the security for each phase in Protocol 1 with the above real vs. ideal model.

The security in **Setup** and **Data Encryption** can be directly obtained from the security of Paillier. We focus on the security analysis of **Computation** and **Evaluation**.

As defined in our *PL-1* privacy, the leakage function for CSP in our Protocol 1 is denoted as

$$\mathcal{L}(\lambda, \bar{x})) = \{q, \textit{supp(q)}, \textit{result}\},$$

where $\bar{x} = \{q, \textit{minup}\}$, q is the mining query given by the Miner and *supp(q)* is mining support of q and *result* is the final

2168-7161 (c) 2017 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCC.2017.2739146, IEEE Transactions on Cloud Computing

Step 1: System Setup. The Evaluator generates the key pair as

$$\{\mathsf{PK}, \mathsf{SK}\} \leftarrow \mathsf{Paillier.KeyGen}(1^{\lambda}).$$

and outputs PK and keeps SK private. Let *minup* denote the minimum support threshold of the mining itemset owned by the Miner (i.e., CSP).

Step 2: Data Encryption. Each user encrypts its own transactions with Paillier encryption and all the transactions from different users are denoted as $T = \{t_1, \cdots, t_m\}$, where $t_i = (e_{i,1}, \cdots, e_{i,n})$ is a transaction.

$$c_{i,j} = [e_{i,j}]_{\mathsf{PK}},$$

where $i \in \{1, \cdots, m\}$ and $j \in \{1, \cdots, n\}$, and then sends $C = \{C_1, \cdots, C_m\}$ to CSP, where each $C_i = (c_{i,1}, \cdots, c_{i,n})$ is the ciphertext of t_i.

Step 3: Computation.

- Once receiving a plaintext mining query $q = (s_1, \cdots, s_n)$ from the Miner, for $i \in \{1, \cdots, m\}$, CSP follows $\mathsf{InnerProPC}$ to compute

$$x_i = [q \circ t_i]_{\mathsf{PK}}.$$

- To preserve the privacy of the mining support from leaking to the Evaluator, CSP selects a *dummy* transactions set $D = \{d_1, \cdots, d_k\}$, where $d_\ell = (d_{\ell,1}, \cdots, d_{\ell,n})$, $d_{\ell,t} \in \{0, 1\}, \ell \in \{1, \cdots, k\}$ and $t \in \{1, \cdots, n\}$, and computes the support of q in the dummy transactions D, denoted as f that is only known to CSP. Then, CSP computes

$$y_\ell = [q \circ d_\ell]_{\mathsf{PK}}.$$

- Then CSP computes

$$\omega_i = \alpha_i(x_i - z), \qquad \omega_{m+\ell} = \alpha_{m+\ell}(y_\ell - z),$$

where $z = [||q||_1]_{\mathsf{PK}}$ and $||q||_1$ is the length of mining query $q = (s_1, \cdots, s_n)$, $\alpha_i \xleftarrow{R} \mathbb{Z}_n, i \in \{1, \cdots, m + k\}$.
- Finally, CSP randomly shuffles $W = \{\omega_1, \cdots, \omega_{m+k}\}$ and sends it to Evaluator.

Step 4: Evaluation.

- Given W, the Evaluator computes

$$\upsilon_i = \begin{cases} 1, & \mathsf{Paillier.Dec}(\mathsf{SK}, \omega_i) = 0, , \\ 0, & \text{otherwise}, \end{cases}$$

and then calculates

$$\upsilon = \sum_{i=1}^{m+k} \upsilon_i.$$

Finally, Evaluator returns υ to CSP.
- Once receiving υ, CSP calculates

$$\mathit{supp}(q) = \upsilon - f,$$

and then outputs 1 if $\mathit{supp}(q) \geq \mathit{minup}$; otherwise 0.

Fig. 3. Details of Protocol 1 with Plaintext Mining.

frequent results. While for the Evaluator, it learns nothing except a dummied support which leaks no private information of the mining itemset. In the following part, we would prove that our Protocol 1 leaks no extra information under semi-honest model except that included in the leakage function.

Computation: This algorithm is run by the CSP to compute encrypted inner products. Given the mining query q, since the transactions are encrypted with Paillier and each inner product is also a ciphertext, then a simulator can simulate those ciphertexts with a same number of randoms. Clearly, if the adversary could distinguish between the view from the real world and the one from the ideal world, then that indicates the adversary could distinguish between a ciphertext of Paillier and a random, which contradicts to the security of Paillier. Therefore, the adversary cannot obtain any private information from the encrypted transactions and inner products in this phase.

Evaluation: In this algorithm, the semi-honest Evaluator collaboratively computes the support of the mining itemset for CSP. We prove that **Evaluation** is secure against a semi-honest adversary $\mathcal{A}^{\mathbb{SH}}_{\text{Evaulator}}$ corrupting the Evaluator in the real world. The view of $\mathcal{A}^{\mathbb{SH}}_{\text{Evaulator}}$ in this phase includes the ciphertexts set $W = \{\omega_1, \cdots, \omega_{m+k}\}$, the decryption of W which includes some 0's and random values, and the output υ which is the number of 0's. Now we build a simulator $\mathcal{S}^{\mathbb{SH}}$ in the ideal world to simulate the view of $\mathcal{A}^{\mathbb{SH}}_{\text{Evaulator}}$ in the real world. $\mathcal{S}^{\mathbb{SH}}$ gets $W' = \{[a_1\alpha'_1], \cdots, [a_{m+k}\alpha'_{m+k}]\}$ where $a_i \xleftarrow{R} \{0, 1\}, \alpha_i \xleftarrow{R} \mathbb{Z}_n$ and decrypts it to get $a_i\alpha'_i, i \in \{1, \cdots, m + k\}$, then calculates υ' which is the number of 0's in $a_i\alpha'_i$. Finally, it returns $\{W', \{a_i\alpha'_i\}_{i\in\{1,\cdots,m+k\}}, \upsilon'\}$ to $\mathcal{A}^{\mathbb{SH}}_{\text{Evaulator}}$.

It is clear that $\mathcal{A}^{\mathbb{SH}}_{\text{Evaulator}}$ is unable to distinguish W and W' due to the semantic security of Paillier. Although $\mathcal{A}^{\mathbb{SH}}_{\text{Evaulator}}$ can decrypt the encrypted W' with its private key, all the $\{a_i\alpha'_i\}_{i\in\{1,\cdots,m+k\}}$ is blinded and randomly distributed as the decrypted W which includes some 0's and random values. For the υ and υ', it is obvious that υ' is randomly distributed due to the randomly choose of a_i and α'_i. υ is also randomly distributed since all the dummy transactions are randomly generated and mixed into the original transactions by CSP in the phase of **Computation**. Therefore, $\mathcal{A}^{\mathbb{SH}}_{\text{Evaulator}}$ is unfeasible to distinguish the ideal world from the real world, that is

$$\left\{\text{IDEAL}_{\Pi,\mathcal{S}^{\mathbb{SH}}}(W)\right\} \stackrel{c}{\equiv} \left\{\text{REAL}_{\Pi,\mathcal{A}^{\mathbb{SH}}_{\text{Evaulator}}}(W)\right\}.$$

In the case that the Evaluator collude with a user (without loss of generality, U_1 with the input transaction T_1), the Evaluator still cannot derive any extra information from the decrypted W and the plaintext T_1 since all the decrypted messages from W are randomized via the random α_i. Therefore, a semi-honest adversary $\mathcal{A}^{\mathbb{SH}}_{\text{Evaluator},U_1}$ corrupting the Evaluator and user U_1 simultaneously is still unable to distinguish from the real world and ideal world, and we have

$$\left\{\text{IDEAL}_{\Pi,\mathcal{S}^{\mathbb{SH}}}(\bar{x})\right\} \stackrel{c}{\equiv} \left\{\text{REAL}_{\Pi,\mathcal{A}^{\mathbb{SH}}_{\text{Evaulator},U_1}}(\bar{x})\right\},$$

where $\bar{x} = \{W, T_1\}$.

Therefore, CSP only learns the last support *supp(q)* and frequent results *result* which are included in the leakage function $\mathcal{L}$ in this phase and the Evaluator learns nothing except a dummied support which is unable to leak any private information about the mining itemset. □

5.2 Protocol 2

To achieve a higher security, we now design our second protocol, which allows the Miner to submit an encrypted mining query. As a result, the CSP and the Evaluator will collaboratively evaluate

2168-7161 (c) 2017 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCC.2017.2739146, IEEE Transactions on Cloud Computing

Step 1: System Setup. The Evaluator generates two key pairs for data encryption and support evaluation as

$$\{\mathsf{PK_{Enc}}, \mathsf{SK_{Enc}}\} \leftarrow \mathsf{BGN.KeyGen}(1^\lambda),$$

$$\{\mathsf{PK_{Eva}}, \mathsf{SK_{Eva}}\} \leftarrow \mathsf{Paillier.KeyGen}(1^\lambda).$$

and then outputs $\mathsf{PK_{Enc}}, \mathsf{PK_{Eva}}$ and keeps $\mathsf{SK_{Enc}}, \mathsf{SK_{Eva}}$ private.

Step 2: Data Processing. Each user U_i encrypts its transactions $t_i = (e_{i,1}, \cdots, e_{i,n})$ with BGN cryptosystem as

$$c_{i,j} = \|e_{i,j}\|_{\mathsf{PK_{Enc}}},$$

where $i \in \{1, \cdots, m\}$ is the number of transactions, and $j \in \{1, \cdots, n\}$ is the dimension of each transaction, and gets $C_i = (c_{i,1}, \cdots, c_{i,n})$, then sends $C = \{C_1, \cdots, C_m\}$ to CSP.

Step 3: Computation.

- Given a mining itemset $q = (s_1, \cdots, s_n)$, the Miner computes
$$r_j = \|s_j\|_{\mathsf{PK_{Enc}}}, \qquad z = \|||q||_1\|_{\mathsf{PK_{Enc}}},$$
and sends $\{Q, z\}$ to CSP, where $Q = (r_1, \cdots, r_n)$ and $||q||_1$ is the length of $q = (s_1, \cdots, s_n)$.
- Similar to Protocol 1, CSP follows InnerProCC to calculate
$$x_i = \|q \circ t_i\|_{\mathsf{PK_{Enc}}}$$
with the inputs Q and C, and then selects a dummy transaction set D to compute
$$y_j = \|q \circ d_j\|_{\mathsf{PK_{Enc}}}$$
with the additive homomorphism of BGN.
- As Protocol 1 in Step 3, CSP calculates a randomized set $W = \{\omega_1, \cdots, \omega_{m+k}\}$ and computes
$$W' = \pi(W)$$
with a secret permutation function π. Finally CSP sends $W' = \{\omega'_1, \cdots, \omega'_{m+k}\}$ to the Evaluator.

Step 4: Evaluation.

- Given W', the Evaluator computes (under Paillier)
$$\upsilon'_i = \begin{cases} [1]_{\mathsf{PK_{Eva}}}, & \mathsf{BGN.Dec}(\mathsf{SK_{Enc}}, \omega'_i) = 0, \\ [0]_{\mathsf{PK_{Eva}}}, & \text{otherwise}, \end{cases}$$
and then returns $V' = \{\upsilon'_1, \cdots, \upsilon'_{m+k}\}$ to CSP.
- Once receiving V', CSP computes
$$V = \pi^{-1}(V'),$$
where $V = \{\upsilon_1, \cdots, \upsilon_{m+k}\}$, and then removes the dummy results and calculates
$$[\mathit{supp}(q)]_{\mathsf{PK_{Eva}}} = \prod_{i=1}^{m} \upsilon_i.$$

Step 5: Comparison. Receiving the encrypted minimum support from the Miner, CSP and Evaluator together follow Compare on two encrypted values $[\mathit{supp}(q)]_{\mathsf{PK_{Eva}}}$ and $[\mathit{minup}]_{\mathsf{PK_{Eva}}}$. Finally, CSP outputs 1 if
$$\mathit{supp}(q) \geq \mathit{minup},$$
otherwise outputs 0.

Fig. 4. Details of Protocol 2 with Ciphertext Mining.

the encrypted mining support via calculating inner products over the encrypted data, and then decide whether a mining query is frequent by evaluating comparison on encrypted data. We leverage InnerProCC and Compare introduced in Sec. 4 to fulfill these tasks.

Details. The details of this second protocol is a little complex, so we describe it as follows:

- **Step 1**: the Evaluator generates key pairs of BGN and Paillier, where BGN is used for encrypting all the private items and Paillier is for evaluating the support of mining itemset in **Step 4**.
- **Step 2**: each user encrypts its own transactions with BGN.
- **Step 3**: the Miner encrypts its mining query with BGN and sends it to CSP. CSP runs InnerProCC to compute each encrypted inner product, and then gets the encrypted dummy inner products by selecting a dummy transaction set the same as Protocol 1. Then CSP randomizes the computed inner products and utilizes a secret permutation function π to permute the randomized inner products W rather than randomly shuffling them as in Protocol 1, and finally sends the permuted W to the Evaluator.
- **Step 4**: the Evaluator and CSP together evaluate the encrypted support of the mining itemset by one-round interaction. Specifically, the Evaluator encrypts different values (i.e., 1 or 0) using Paillier according to the results decrypted from the randomized inner products and then sends the encrypted values V' to CSP, then CSP removes the dummy values derived from the dummy transactions to get the encrypted real support of the mining itemset by using Paillier .
- **Step 5**: once receiving the encrypted minimum support threshold from the Miner, CSP and the Evaluator run the comparison protocol Compare on the encrypted support and threshold, then output 1 if the mining itemset is frequent and otherwise output 0.

As we mentioned, the mining aims of Protocol 2 are that the Miner obtains the last mining result, and neither CSP nor Evaluator can get any private information about the mining itemset except the frequent result 0 or 1. We analyze the security of our second protocol under semi-honest model similar to Protocol 1. The security theorem under semi-honest model and the detailed proof are illustrated as below. The security of this protocol is similar as the *database privacy* as in [10].

Theorem 2. *Our Protocol 2 is secure against the semi-honest CSP or Evaluator with PL-3 privacy as long as both BGN and Paillier are semantically secure and non-collusion is allowed between CSP and Evaluator.*

Proof: Similar to the Protocol 1, we also adopt the simulation-based approach to prove the security of Protocol 2. Specifically, the security of **Setup** and **Data Encryption** can be easily deduced into the security of BGN and Paillier the same as in Theorem 1. For **Computation**, **Evaluation** and **Comparison**, similar to the prove of Theorem 1, the leakage function in these phases for CSP and the Evaluator with *PL-3* privacy is denoted as

$$\mathcal{L}(\lambda, \bar{x})) = \{\mathit{result}\},$$

where $\bar{x} = \{\|q\|, \|\mathit{minup}\|\}$.

We can easily prove the security of the **Computation** in a similar way as in Theorem 1. The only difference is that CSP receives the encrypted mining request $\{Q, z\}$ from the Miner and can collude with the users to derive the private information of the mining request. Due to the semantic security of BGN, a

2168-7161 (c) 2017 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCC.2017.2739146, IEEE Transactions on Cloud Computing

semi-honest adversary $\mathcal{A}^{\mathbb{SH}}_{\text{CSP}}$ corrupting CSP has no advantage to distinguish Q and z from the real world and the ideal world. Therefore, we have

$$\left\{\text{IDEAL}_{\Pi,\mathcal{S}^{\mathbb{SH}}}(\bar{x})\right\} \stackrel{c}{\equiv} \left\{\text{REAL}_{\Pi,\mathcal{A}^{\mathbb{SH}}_{\text{CSP}}}(\bar{x})\right\},$$

where $\bar{x} = \{Q, z\}$.

For the case of the collusion between CSP and a user (e.g., U_1 with the input transaction T_1) in **Computation**, the adversary $\mathcal{A}^{\mathbb{SH}}_{\text{CSP},U_1}$ corrupting CSP and U_1 at the same time still cannot obtain any information about other user's input (e.g., U_2 with input T_2 without loss of generality), because it only has a negligible advantage (i.e., the advantage introduced by two message/ciphertext pairs $\{T_1, \|T_1\|\}$ and $\{\langle I, T_1\rangle, \|\langle I, T_1\rangle\|)\}$ in the semantic security game of BGN to distinguish the ciphertexts $\{|T_2\|, \|\langle I, T_2\rangle\|\}$ between the ideal world and real world under the input of $\{I, T_1, \|T_2\|\}$ in the above simulation. So, we can conclude

$$\left\{\text{IDEAL}_{\Pi,\mathcal{S}^{\mathbb{SH}}}(\bar{x})\right\} \stackrel{c}{\equiv} \left\{\text{REAL}_{\Pi,\mathcal{A}^{\mathbb{SH}}_{\text{CSP},U_1}}(\bar{x})\right\},$$

where $\bar{x} = \{I, T_1, \|T_2\|\}$.

Therefore, except what is involved in the leakage function $\mathcal{L}$, CSP learns no extra private information of the encrypted transactions and mining queries.

Evaluation: This algorithm is collaboratively conducted by the non-colluding CSP and Evaluator, thus we need to prove that the **Evaluation** is secure not only against a semi-honest adversary $\mathcal{A}^{\mathbb{SH}}_{\text{CSP}}$ corrupting CSP in the real world but also against a semi-honest adversary $\mathcal{A}^{\mathbb{SH}}_{\text{Evaluator}}$ corrupting the Evaluator respectively. For security proof against the semi-honest $\mathcal{A}^{\mathbb{SH}}_{\text{Evaluator}}$ and the case of collusion between CSP and a user, we can use the similar proof way as in Theorem 1. Here we just need to prove that it is secure against a semi-honest $\mathcal{A}^{\mathbb{SH}}_{\text{CSP}}$ who maybe also collude with the data user.

Obviously, the view of $\mathcal{A}^{\mathbb{SH}}_{\text{CSP}}$ in **Evaluation** includes $\{V', [\textit{supp(q)}]\}$ which are encrypted with the Evaluator's public key under Paillier. Since $\mathcal{A}^{\mathbb{SH}}_{\text{CSP}}$ has no knowledge of the private key of the Evaluator, if $\mathcal{A}^{\mathbb{SH}}_{\text{CSP}}$ can distinguish the real world and the ideal world from $\{V', [\textit{supp(q)}]\}$, then there must exist an algorithm to distinguish the ciphertexts generated by Paillier, which contradicts to the assumption that Paillier has semantic security. Hence, $\mathcal{A}^{\mathbb{SH}}_{\text{CSP}}$ is computationally indistinguishable from the ideal world and the real world, and we have

$$\left\{\text{IDEAL}_{\Pi,\mathcal{S}^{\mathbb{SH}}}(\bar{x})\right\} \stackrel{c}{\equiv} \left\{\text{REAL}_{\Pi,\mathcal{A}^{\mathbb{SH}}_{\text{CSP}}}(\bar{x})\right\},$$

where $\bar{x} = \{V', [\textit{supp(q)}]\}$.

For the case of the collusion between CSP and a user (e.g., U_1 with input transaction T_1), the additional view of the adversary $\mathcal{A}^{\mathbb{SH}}_{\text{CSP},U_1}$ corrupting CSP and U_1 simultaneously is the input T_1. The same as the above, $\mathcal{A}^{\mathbb{SH}}_{\text{CSP},U_1}$ is unable to refer to any extra private information from $\{T_1, V', [\textit{supp(q)}]\}$ due to the semantic security of Paillier. Therefore, we have

$$\left\{\text{IDEAL}_{\Pi,\mathcal{S}^{\mathbb{SH}}}(\bar{x})\right\} \stackrel{c}{\equiv} \left\{\text{REAL}_{\Pi,\mathcal{A}^{\mathbb{SH}}_{\text{CSP},U_1}}(\bar{x})\right\},$$

where $\bar{x} = \{T_1, V', [\textit{supp(q)}]\}$.

Hence, the CSP or the Evaluator in this phase are unable to learn any private information from all these intermediate encrypted results and blinded values.

Comparison: The CSP and the Evaluator collaboratively complete the computation in this phase without the collusion. The view of CSP includes two inputs $\{[\textit{supp(q)}], [\textit{minup}]\}$ encrypted by the Evaluator's public key under Paillier. Similar to the security proof of **Evaluation**, the adversary $\mathcal{A}^{\mathbb{SH}}_{\text{CSP}}$ is unable to distinguish the real world from the ideal world and we have

$$\left\{\text{IDEAL}_{\Pi,\mathcal{S}^{\mathbb{SH}}}(\bar{x})\right\} \stackrel{c}{\equiv} \left\{\text{REAL}_{\Pi,\mathcal{A}^{\mathbb{SH}}_{\text{CSP}}}(\bar{x})\right\},$$

where $\bar{x} = \{[\textit{supp(q)}], [\textit{minup}]\}$.

For the semi-honest adversary $\mathcal{A}^{\mathbb{SH}}_{\text{Evaluator}}$ corrupting the Evaluator, its view in this phase is $[r \cdot (\textit{supp(q)} - \textit{minup})]$ encrypted with its own public key. After decryption, $\mathcal{A}^{\mathbb{SH}}_{\text{Evaluator}}$ can view a randomized value $r \cdot (\textit{supp(q)} - \textit{minup})$. To illustrate that the semi-honest adversary $\mathcal{A}^{\mathbb{SH}}_{\text{Evaluator}}$ is unable to learn private information from its view, we build a corresponding simulator $\mathcal{S}^{\mathbb{SH}}$ in the ideal world. Suppose $\mathcal{S}^{\mathbb{SH}}$ has an input $[r' \cdot (a - b)]$ encrypted under Paillier, where r' is randomly chosen from the same domain as r. Without loss of generality, we can suppose $a = 1$ and $b = 2$. Then, $\mathcal{S}^{\mathbb{SH}}$ decrypts it to get $r' \cdot (a - b)$. As introduced in Subsection 4.4, the probability of correctly guessing r could be extremely small since it was randomly chosen from a very large space (e.g., 480-bits space). Therefore, $\mathcal{S}^{\mathbb{SH}}$ is unable to distinguish $r \cdot (\textit{supp(q)} - \textit{minup})$ from $r' \cdot (a - b)$. That is to say, $\mathcal{A}^{\mathbb{SH}}_{\text{Evaluator}}$ is computationally indistinguishable its view from the real world and the ideal world, and we have

$$\left\{\text{IDEAL}_{\Pi,\mathcal{S}^{\mathbb{SH}}}(\bar{x})\right\} \stackrel{c}{\equiv} \left\{\text{REAL}_{\Pi,\mathcal{A}^{\mathbb{SH}}_{\text{Evaluator}}}(\bar{x})\right\},$$

where $\bar{x} = \{[r \cdot (\textit{supp(q)} - \textit{minup})]\}$.

Therefore, except the final frequent results which are involved in the leakage function $\mathcal{L}$, no extra private information is leaked to the CSP or the Evaluator in this phase.

□

Discussion. In Protocol 1, CSP can locally compute the support f of the mining itemset q in the dummy transactions and then easily calculates the real support of q in the original transactions contributed by all the users. However, in our Protocol 2, the mining itemset is encrypted and CSP is not able to learn any information about q as well as the dummy support f. Hence, we use a secret permutation function π to permute the encrypted inner products computed from the dummy and original transactions. Once the Evaluator returns the encrypted results, CSP computes the inverse of them with π, and thus it can correctly remove all the dummy values and calculate the encrypted real support of the mining itemset.

Another issue in the phase of **Evaluation** is, the evaluator needs to run $\mathsf{BGN.Dec}(\mathsf{SK}_{\mathsf{Enc}}, \omega'_i)$ to decrypt. Normally, the size of messages in BGN is assumed to small, so that the decryption is still possible by solving discrete log. While in our case, we only care about the messages with 0s but do not care about others, so the decryption will be efficient. Specifically, in the decryption algorithm of BGN, the first step for decryption is computing $C^{q_1} = (g^{q_1})^m$, then it suffices to compute the discrete log of C^{q_1} base g^{q_1} to recover the message m. In our protocol, it is enough to compute the first step, that is if $C^{q_1} = 1$ (i.e., $\omega'_i = 0$), then the Evaluator calculates $\mathsf{Paillier.Enc}(\mathsf{PK}_{\mathsf{Eva}}, 1)$; otherwise, that is $C^{q_1} \neq 1$ (i.e., $\omega'_i \neq 0$), the Evaluator calculates $\mathsf{Paillier.Enc}(\mathsf{PK}_{\mathsf{Eva}}, 0)$.

5.3 Protocol 3 with Enhanced Efficiency

With the above protocols, we can easily discover all the frequent itemsets using the same idea as the classic A-Priori algorith-

2168-7161 (c) 2017 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCC.2017.2739146, IEEE Transactions on Cloud Computing

m [35], which can successfully reduce the number of the possible candidate mining itemsets, at the expense of performing multiple passes over the transactions rather than one pass. Specifically, if we want to mine all the frequent item pairs, the first pass is to find out all the frequent single items, and based on the *monotonicity* of itemsets, a frequent item pair must be composed with two frequent items. Thus, the number of the candidate mining pairs would be significantly reduced with a sufficiently high support threshold.

In our Protocol 2, it would be less efficient to mine a great number of itemsets since each mining itemset needs to be extended into an n-length vector and CSP needs to do many unnecessary operations for computing inner products. To optimize these unnecessary computations, we first anonymize the public attribute list so that only the Miner and data users learn the original attribute list. As a result, when requesting a mining, the Miner does not need to encrypt it and just directly sends it to CSP as in Protocol 1. In such case, due to the anonymization of the corresponding attribute list, CSP is not able to know the private mining items except the number of the items in the mining itemset. After receiving the plaintext mining vector, CSP only needs to find the positions with 1 in the vector, and then computes encrypted items in the transactions with the same positions. Specifically, suppose an encrypted transaction is $C = (c_1, c_2, c_3, c_4)$ and a mining vector is $q = (1, 0, 0, 1)$, CSP just selects c_1 and c_4, and computes $c_1 \cdot c_4$ which is the addition of the corresponding plaintexts. Obviously, it does not need any multiplication computations of the plaintexts, so it is enough to use Paillier rather than BGN. Through this optimization, we can significantly improve the time of computing encrypted inner products and enhance the efficiency of the entire protocol. Note that we still need to preserve the sensitive support from CSP and Evaluator, so the rest operations for evaluating the mining support are the same as described in Protocol 2. The security is defined as below and the detailed proof is similar to Theorem 1 and Theorem 2.

Theorem 3. *Our Protocol 3 is secure against the semi-honest CSP with PL-2 privacy as long as Paillier encryption is semantically secure, and also against a semi-honest Evaluator to distinguish the intermediate results of the computation for the mining itemsets as long as non-collusion is occurred between CSP and Evaluator.*

Proof: Our Protocol 3 has a *PL-3* privacy and according to the definition in Subsection 3.3, the leakage function with *PL-3* privacy is denoted as

$$\mathcal{L}(\lambda, \bar{x})) = \{itemnum(q), result\},$$

where $\bar{x}$ includes the anonymized q and the minimum threshold *minup* encrypted with Pillier, and *itemnum(q)* which is the number of items in the mining itemset.

Even through the mining query is not encrypted, the anonymization preserves it from leaking to the CSP except the number of items *itemnum(q)* which is involved in the leakage function $\mathcal{L}$. For the proof of left part in **Computation**, **Evaluation** and **Comparison**, it is the same with in Theorem 2 since Protocol 3 is optimized based on Protocol 2 and we neglect its detailed analysis here.

□

TABLE 2
Computation time (in millisecond) on different operations

Paillier	An exponentiation in $\mathbb{Z}^*_{N^2}$ ($\mathsf{Expon}_{\mathsf{Paillier}}$)	0.006
	A multiplication in $\mathbb{Z}^*_{N^2}$ ($\mathsf{Multi}_{\mathsf{Paillier}}$)	0.0005
BGN	A multiplication in $\mathbb{G}$ of BGN ($\mathsf{Multi}_{\mathsf{BGN},\mathbb{G}}$)	0.02
	A multiplication in $\mathbb{G}_1$ of BGN ($\mathsf{Multi}_{\mathsf{BGN},\mathbb{G}_1}$)	0.009
	An exponentiation in $\mathbb{G}_1$ of BGN ($\mathsf{Expon}_{\mathsf{BGN},\mathbb{G}_1}$)	0.225
	A pairing operation of BGN ($\mathsf{Pairing}_{\mathsf{BGN}}$)	0.42

6 Performance Evaluation

In this section, we evaluate the performance of our protocols, and compare it with previous solutions. The performance evaluations for our privacy-preserving frequent itemset mining protocols are tested on four desktop computers running Windows 7 with Intel Core i5-4570 3.20 GHz CPU and 8 GB RAM. Specifically, one is used to simulate data users to encrypt all the transactions and upload them to CSP; one is used as the Miner, and the rest of the two are leveraged as the CSP and the Evaluator respectively. To inflate the network latency, we suppose that the delay of one round interaction is around 10 milliseconds on average, and the transmission rate between two servers is around 10 Mbps.

We first implement our three protocols for single itemset mining. To achieve the security requirement, we choose $|N| = 1024$ in Paillier and $|p| = 160$ bits with at least 1024-bit security with Type A pairing in BGN. The computation time on different fundamental cryptographic operations in our implementation are presented in Table 2.

6.1 Efficiency of Our Protocols

The performance of our three protocols are presented in two parts, including data encryption-uploading phase (i.e., **Data Encryption**) and support computation-evaluation phase (including **Computation**, **Evaluation** and an extra **Comparison** in the second protocol). The number of transactions is denoted by m, and the number of dimensions in each transaction is n.

Data Encryption-Uploading Phase. Note that the data encryption in the first phase can be done off-line before launching mining request. The performance of the phase of data encryption-uploading for three protocols are shown in Table 3. We can observe that encrypting one thousand transactions with set dimensions of 20 takes 2.41 seconds in Protocol 1 and it requires around 6 seconds for the dimensions of 50. It is obvious that our Protocol 3 costs the same time with the Protocol 2 in this phase since both two protocols adopt BGN cryptosystem, and each of them takes average 2.94 seconds to encrypt one thousand transactions with dimension of 20 and 7.48 seconds with dimension of 50.

Support Computation-Evaluation Phase. In Protocol 1, CSP launches association rule mining and the only interaction is the one-round interaction with the Evaluator to compute the support of the mining itemset. Once recovering the mining support, CSP can efficiently obtain the frequent itemsets by comparing two unencrypted integers. Hence, our first protocol is extremely efficient. As shown in Table 4, the running time in this phase is linear to the size of transactions and set dimensions. In Protocol 1, it only costs around 1 second to evaluate the frequent itemset over one thousand transactions with $n = 50$, while it takes almost 34 seconds with the same scale size in Protocol 2. This is because that all the computation and comparison are operated over encrypted data, which requires more time. Moreover, it is clear to see from Table 4 that the communication cost in Protocol 2 is over two times

2168-7161 (c) 2017 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCC.2017.2739146, IEEE Transactions on Cloud Computing

TABLE 3
Performance of the phase of data encryption-uploading, which can be previously done off-line. Dim. denotes the dimension of each transaction. Trans. denotes the number of transactions in the database. Time is in second and communication cost is in MB.

Dim. n	Trans. m	Protocol 1 Time (s)	Protocol 1 Comm. (MB)	Protocol 2 & Protocol 3 Time (s)	Protocol 2 & Protocol 3 Comm. (MB)
20	100	0.23	0.244	0.30	0.244
	1000	2.41	2.44	2.94	2.44
50	100	0.61	0.61	0.75	0.61
	1000	6.05	6.10	7.48	6.10

TABLE 4
Performance of the phase of support computation-evaluation, where we suppose that the size of dummy transactions $k = m/2$. Dim. denotes the dimension of each transaction. Trans. denotes the number of transactions in the database. Time denotes the total running time of the protocol. Comm. denotes the communication cost of the protocol. Here the time is in second and communication cost is in MB.

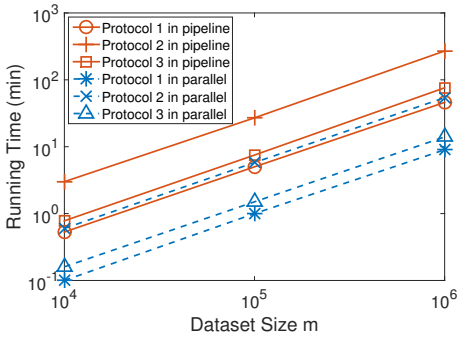
Dim. n	Trans. m	Protocol 1 Cost. CSP	Evaluator	Time	Comm.	Protocol 2 Cost. CSP	Evaluator	Time	Comm.	Protocol 3 Cost. CSP	Evaluator	Time	Comm.
20	100	0.02	0.06	0.18	0.0183	1.40	0.05	1.51	0.0368	0.12	0.047	0.35	0.0368
	1000	0.21	0.59	0.99	0.183	13.4	0.49	14.46	0.368	1.32	0.46	1.97	0.36
50	100	0.035	0.06	0.19	0.0183	3.32	0.052	3.49	0.0368	0.046	0.058	0.23	0.0368
	1000	0.36	0.62	1.04	0.183	32.9	0.54	33.84	0.368	0.45	0.56	1.12	0.368

than that in Protocol 1, and this demonstrates that our Protocol 2 needs more computation and communication cost than Protocol 1. As we known that it is a necessary tradeoff between efficiency and privacy. Furthermore, with our enhancement of Protocol 2, the efficiency is significant improved in our Protocol 3 as shown in Table 4.

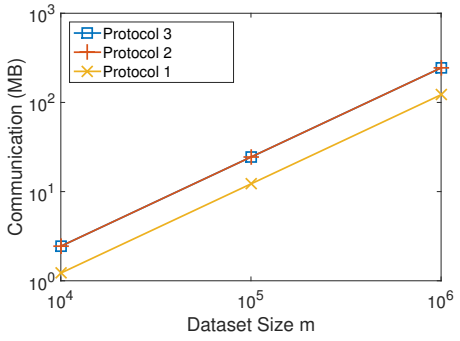
We test the performance of three protocols for single mining itemset query with the number of sample transactions varying from 10 thousands to 1 million, where the number of dimensions $n = 20$. In the following implementation, two running modes are utilized for testing the running time: the pipeline mode and parallel mode, which are also used in several other work [25]. All the parallelized experiments are run on a machine with 8 vCPUs and 8 GB RAM. Our results show that the parallel mode on multi-core machines can significantly optimize the performance, since all the dominant computations in our protocols are encrypting data and evaluating inner products, which can be simultaneously evaluated in parallel.

As shown in Fig. 5(a), the performance of three protocols running in the parallel mode can be at least five times faster than that in the pipeline mode. For instance, if the transactions set size is 1 million, it requires around 42 minutes to run Protocol 1 in the pipeline mode; while with the parallel mode, the running time can be deduced to only 8 minutes on average with the same scale set size. Similarly, the running time of Protocol 2 and Protocol 3 can also be optimized to less than 54 minutes and 14 minutes respectively with one million scale under parallel mode. Furthermore, we can see that the communication overhead of our protocols is also linear to the size of transactions set as shown in Fig. 5(b).

Note that in the above implementation, the change of the number of the items in the mining query has an extremely slight effect on our Protocol 3; it is very fast for the multiplication operations over the ciphertexts under Paillier as presented in Table 2.



(a) Comparison of running time in two modes



(b) Comparison of communication overhead

Fig. 5. Computation and communication cost of our three protocols under different running modes, the transactions set size varied from 10 thousands to 1 million and the dummy transactions set size $k = m/2$.

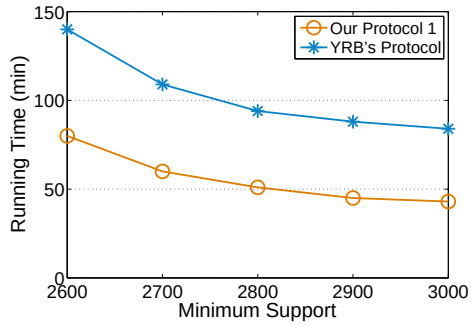
6.2 Efficiency Comparison

For efficiently mining all the possible frequent itemsets and rules, we test the whole mining performance achieved from our Protocol 1 and Protocol 3 separately. The most recent two works are introduced in [8] and [10], while the model presented by [8] was not suitable for our cloud-based system model. Therefore, we focus on the comparison of our protocols with association rule mining protocols presented by Yi *et al.* [10]. To be consistent with [10], we also use the same chess database[3] as our transaction database, which totally has 3,196 transactions and 74 possible attributes. We run Yi's coding on the same experimental environment as ours, which is Windows 7 with Intel Core i5-4570 3.20 GHz CPU and 8 GB RAM. The performance of the comparison is evaluated with various minimum support thresholds and divided with two different security levels: item privacy and database privacy. Fig. 6(a) presents the efficiency of our Protocol 1 and YBR's protocol with item privacy, and Protocol 3 is compared

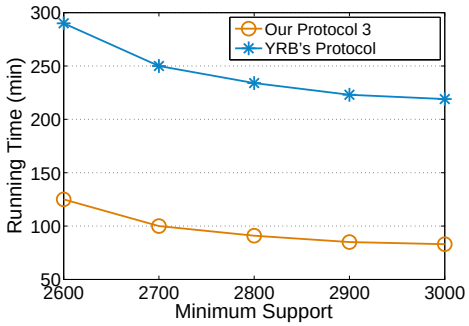
3. http://fimi.ua.ac.be/data/

2168-7161 (c) 2017 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCC.2017.2739146, IEEE Transactions on Cloud Computing



(a) Item Privacy



(b) Database Privacy

Fig. 6. Comparison the performance between our protocols and YRB's protocols with different security levels, where the size of dummy transactions in both two protocols are $m/2$.

with YBR's protocol with database privacy as shown in Fig. 6(b). It is obvious that our protocols are more efficient than YBR's Protocols under the same settings and the same security level. Additionally, we still can significantly improve the performance by running under parallel mode as in Fig. 5(a) when mining on a large scale dataset.

7 Conclusion and Future Work

In this paper, we propose three practical privacy-preserving frequent itemset mining protocols on encrypted cloud data. Our Protocol 1 achieves an extremely higher mining performance while our Protocol 2 provides a stronger privacy guarantee. Toward more practical efficiency, we optimize the performance of our second protocol by leveraging a minor leakage of privacy to achieve our Protocol 3. Experimental results show that our protocols are much more efficient than previous work with the same security levels. In our future work, we focus on further improving the efficiency of frequent itemset mining on larger scale database. Furthermore, due to the special properties of streaming data (real-time and continuous) and its extensively practical application, we focus on the study of the efficient privacy-preserving frequent itemset mining algorithm on stream data.

References

[1] S. Brin, R. Motwani, J. D. Ullman, and S. Tsur, "Dynamic itemset counting and implication rules for market basket data," in *ACM SIGMOD Record*, vol. 26, no. 2. ACM, 1997, pp. 255–264.

[2] W. Lee and S. J. Stolfo, "Data mining approaches for intrusion detection," in *Usenix Security*, 1998.

[3] C. Estan and G. Varghese, *New directions in traffic measurement and accounting*. ACM, 2002, vol. 32, no. 4.

[4] B. Mobasher, N. Jain, E.-H. Han, and J. Srivastava, "Web mining: Pattern discovery from world wide web transactions," Technical Report TR96-050, Department of Computer Science, University of Minnesota, Tech. Rep., 1996.

[5] J. Vaidya and C. Clifton, "Privacy preserving association rule mining in vertically partitioned data," in *Proceedings of the eighth ACM SIGKDD international conference on Knowledge discovery and data mining*. ACM, 2002, pp. 639–644.

[6] M. Kantarcioglu and C. Clifton, "Privacy-preserving distributed mining of association rules on horizontally partitioned data," *IEEE Transactions on Knowledge & Data Engineering*, no. 9, pp. 1026–1037, 2004.

[7] J. Vaidya and C. Clifton, "Secure set intersection cardinality with application to association rule mining," *Journal of Computer Security*, vol. 13, no. 4, pp. 593–622, 2005.

[8] T. Tassa, "Secure mining of association rules in horizontally distributed databases," *Knowledge and Data Engineering, IEEE Transactions on*, vol. 26, no. 4, pp. 970–983, 2014.

[9] C. Dong and L. Chen, "A fast secure dot product protocol with application to privacy preserving association rule mining," in *Advances in Knowledge Discovery and Data Mining*. Springer, 2014, pp. 606–617.

[10] X. Yi, F.-Y. Rao, E. Bertino, and A. Bouguettaya, "Privacy-preserving association rule mining in cloud computing," in *Proceedings of the 10th ACM Symposium on Information, Computer and Communications Security*. ACM, 2015, pp. 439–450.

[11] R. Agrawal, T. Imieliński, and A. Swami, "Mining association rules between sets of items in large databases," in *ACM SIGMOD Record*, vol. 22, no. 2. ACM, 1993, pp. 207–216.

[12] R. Agrawal, R. Srikant *et al.*, "Fast algorithms for mining association rules," in *Proc. 20th int. conf. very large data bases, VLDB*, vol. 1215, 1994, pp. 487–499.

[13] A. Evfimievski, R. Srikant, R. Agrawal, and J. Gehrke, "Privacy preserving mining of association rules," *Information Systems*, vol. 29, no. 4, pp. 343–364, 2004.

[14] S. R. Oliveira and O. R. Zaiane, "Privacy preserving frequent itemset mining," in *Proceedings of the IEEE international conference on Privacy, security and data mining-Volume 14*. Australian Computer Society, Inc., 2002, pp. 43–54.

[15] Y. Saygin, V. S. Verykios, and A. K. Elmagarmid, "Privacy preserving association rule mining," in *Research Issues in Data Engineering: Engineering E-Commerce/E-Business Systems, 2002. RIDE-2EC 2002. Proceedings. Twelfth International Workshop on*. IEEE, 2002, pp. 151–158.

[16] S. J. Rizvi and J. R. Haritsa, "Maintaining data privacy in association rule mining," in *Proceedings of the 28th international conference on Very Large Data Bases*. VLDB Endowment, 2002, pp. 682–693.

[17] A. Evfimievski, R. Srikant, R. Agrawal, and J. Gehrke, "Privacy preserving mining of association rules," *Information Systems*, vol. 29, no. 4, pp. 343–364, 2004.

[18] F. Giannotti, L. V. Lakshmanan, A. Monreale, D. Pedreschi, and H. Wang, "Privacy-preserving mining of association rules from outsourced transaction databases," *Systems Journal, IEEE*, vol. 7, no. 3, pp. 385–395, 2013.

[19] K. Sathiyapriya and G. S. Sadasivam, "A survey on privacy preserving association rule mining," *International Journal of Data Mining & Knowledge Management Process*, vol. 3, no. 2, pp. 119–131, 2013.

[20] R. Kharat, M. Kumbhar, and P. Bhamre, "Efficient privacy preserving distributed association rule mining protocol based on random number," in *Intelligent Computing, Networking, and Informatics*. Springer, 2014, pp. 827–836.

[21] W. K. Wong, D. W. Cheung, E. Hung, B. Kao, and N. Mamoulis, "Security in outsourcing of association rule mining," in *Proceedings of the 33rd international conference on Very large data bases*. VLDB Endowment, 2007, pp. 111–122.

[22] C.-H. Tai, P. S. Yu, and M.-S. Chen, "k-support anonymity based on pseudo taxonomy for outsourcing of frequent itemset mining," in *Proceedings of the 16th ACM SIGKDD international conference on Knowledge discovery and data mining*. ACM, 2010, pp. 473–482.

[23] M. Kantarcioglu, R. Nix, and J. Vaidya, "An efficient approximate protocol for privacy-preserving association rule mining," in *Advances in Knowledge Discovery and Data Mining*. Springer, 2009, pp. 515–524.

[24] L. Qiu, Y. Li, and X. Wu, "Preserving privacy in association rule mining with bloom filters," *Journal of Intelligent Information Systems*, vol. 29, no. 3, pp. 253–278, 2007.

[25] C. Dong, L. Chen, and Z. Wen, "When private set intersection meets big data: an efficient and scalable protocol," in *Proceedings of the 2013 ACM SIGSAC conference on Computer & communications security*. ACM, 2013, pp. 789–800.

2168-7161 (c) 2017 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCC.2017.2739146, IEEE Transactions on Cloud Computing

[26] X. Ge, L. Yan, J. Zhu, and W. Shi, "Privacy-preserving distributed association rule mining based on the secret sharing technique," in *Software Engineering and Data Mining (SEDM), 2010 2nd International Conference on*. IEEE, 2010, pp. 345–350.

[27] J. Lai, Y. Li, R. H. Deng, J. Weng, C. Guan, and Q. Yan, "Towards semantically secure outsourcing of association rule mining on categorical data," *Information Sciences*, vol. 267, pp. 267–286, 2014.

[28] V. Nikolaenko, U. Weinsberg, S. Ioannidis, M. Joye, D. Boneh, and N. Taft, "Privacy-Preserving Ridge Regression on Hundred of Millions of Records," in *Proc. of IEEE S&P'13*, 2013.

[29] A. Peter, E. Tews, and S. Katzenbeisser, "Efficiently Outsourcing Multiparty Computation Under Multiple Keys," *IEEE Transactions on Information Forensics and Security*, vol. 8, no. 12, pp. 2046–2058, 2013.

[30] R. Bost, R. A. Popa, S. Tu, and S. Goldwasser, "Machine learning classification over encrypted data," *Crypto ePrint Archive*, 2014.

[31] R. Curtmola, J. A. Garay, S. Kamara, and R. Ostrovsky, "Searchable Symmetric Encryption: Improved Definitions and Efficient Constructions," in *Proc. of ACM CCS'06*, 2006.

[32] O. Goldreich, *Foundations of cryptography: volume 2, basic applications*. Cambridge university press, 2004.

[33] P. Paillier, "Public-key cryptosystems based on composite degree residuosity classes." Springer, 1999, pp. 223–238.

[34] D. Boneh, E.-J. Goh, and K. Nissim, "Evaluating 2-dnf formulas on ciphertexts," in *Theory of cryptography*. Springer, 2005, pp. 325–341.

[35] J. Han, M. Kamber, and J. Pei, *Data mining: concepts and techniques: concepts and techniques*. Elsevier, 2011.

2168-7161 (c) 2017 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.